

RESOLUTION NUMERIQUE



SYNTHESE

*Algorithmes
de calcul numérique*

Algorithmes de calcul numérique

- Solution d'un système stationnaire par les méthodes de **dichotomie** et **Newton** ;
- **Dérivation** et **intégration numérique** ;
- Résolution d'équation différentielle par la méthode du **schéma explicite d'Euler** ;

Afin de gagner en efficacité, nous utiliserons allègrement le module **Numpy**. De même, nous pourrons tracer une image de notre travail à l'aide du module **Matplotlib**.

1. ZERO D'UNE FONCTION : DICHOTOMIE, NEWTON

Problématique :

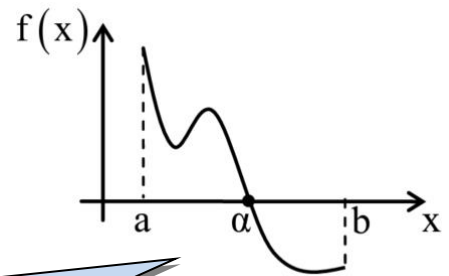
Soit f une fonction continue sur un intervalle $[a, b]$ de $\mathbb{R}$.

Nous cherchons par exemple $\alpha \in [a, b]$ tel que $f(\alpha) = 0$.

Critères d'arrêt :

Soit ε une tolérance fixée. On utilise, soit :

- un critère basé sur l'incrément : $|X_{n+1} - X_n| < \varepsilon$;
- un critère basé sur le résidu : $|f(X_n)| < \varepsilon$.



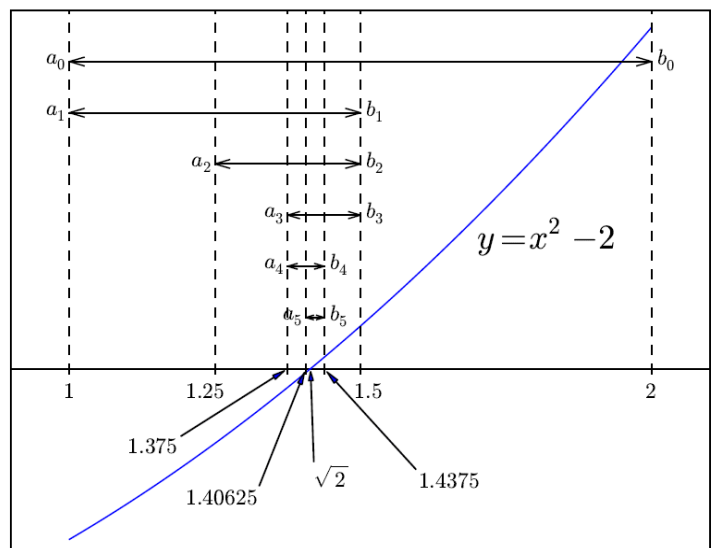
Ces méthodes peuvent se généraliser à toute valeur α mais ne pourront s'appliquer que si α est unique dans l'intervalle $[a, b]$!

1.1 Principe de la méthode de dichotomie :

Dans le cas où l'on recherche un zéro, le principe de la dichotomie est d'encadrer de plus en plus finement un zéro possible, sa présence étant détectée par un changement de signe de la fonction, supposée continue.

Cette méthode est illustrée ci-contre...

Indice : La commande : `assert f(a)*f(b)<=0` vérifie que la fonction change bien de signe entre a et b et que ε est strictement positif...



Son principe de calcul est détaillé ci-dessous :

- On initialise l'encadrement par $a_0 < b_0$ tel que f change de signe entre ces valeurs ;
- On coupe l'intervalle en deux en son milieu. Puisqu'il y a changement de signe sur l'intervalle, il y a changement de signe sur l'une des deux moitiés. On conserve la moitié correspondante ;
- On continue de la sorte jusqu'à ce que l'intervalle obtenu soit suffisamment petit pour donner une valeur approchée du zéro à la marge de l'erreur ε souhaitée.

1.2 Proposition d'algorithme de dichotomie sous python :

```
@author: Goncalves David
"""

def zero_dicho(f,a,b,epsi):
    assert f(a)*f(b) <= 0 and epsi>0
    g,d=a,b #permet de conserver les bornes initiales

    while d-g>2*epsi:
        m=(g+d)/2.
        if f(g)*f(m) <= 0:
            d=m # on decale la borne de droite
        else:
            g=m # on decale la borne de gauche
    return m
```

1.3 Principe de la méthode de Newton :

La méthode de Newton est plus efficace... à condition d'assurer sa convergence !

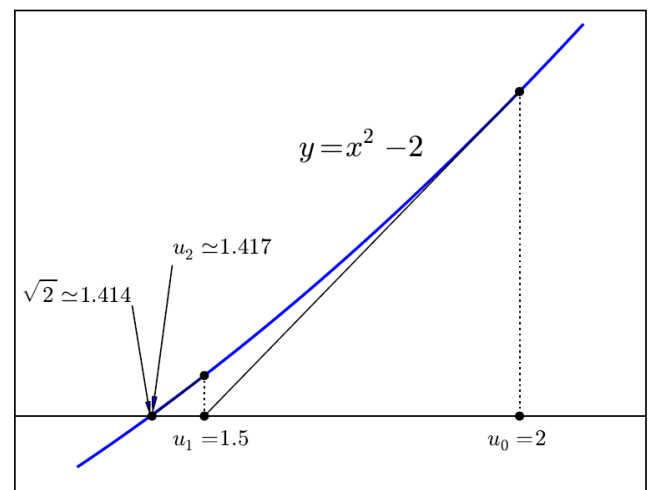
En suppose f dérivable, on part d'une valeur initiale u_0 et on suit la tangente. L'intersection de cette tangente avec l'axe des abscisses donne une nouvelle valeur u_1 . On reprend la tangente depuis l'abscisse u_1 et ainsi de suite...

On a alors la suite : $u_{n+1} = u_n - \frac{f(u_n)}{f'(u_n)}$

Cette méthode est illustrée ci-contre...

Et le principe du calcul détaillé ci-dessous :

- On part d'une valeur initiale u_0 ;
- On construit u_1 comme l'intersection de la tangente en u_0 et de l'axe des abscisses ;
- On réitère cette construction de façon à donner une valeur approchée du zéro à la marge de l'erreur ϵ souhaitée.



1.3 Proposition d'algorithme de Newton sous python :

```
@author: Goncalves David
"""

def sol_newton(f,g,u0,epsi):
    u=u0

    while f(u)>=abs(epsi):
        u=u-f(u)/g(u)
    return u
```

1.4 Application : trouver le $T_{R5\%}$ d'un second ordre apériodique :

Afin de donner une illustration concrète à ces méthodes tout en restant dans des calculs simples, on se propose de trouver le temps de réponse à 5% pour un système du second ordre soumis à un échelon de Heaviside...

Pour mémoire, la réponse en Laplace peut alors s'écrire dans ce cas :

$$S(p) = \frac{K.e_0}{p(1+\tau_1.p)(1+\tau_2.p)} = \frac{K.e_0}{p(1+(\tau_1+\tau_2).p+\tau_1.\tau_2.p^2)}$$

Et la réponse temporelle est : $s(t) = K.e_0 \left(1 - \frac{1}{\tau_1 - \tau_2} \left(\tau_1 \cdot e^{-\frac{t}{\tau_1}} - \tau_2 \cdot e^{-\frac{t}{\tau_2}} \right) \right)$

Proposition d'algorithme

```
@author: Goncalves David
"""

def sol_dicho(f,a,b,epsi):
    assert f(a)*f(b) <= 0 and epsi>0
    g,d=a,b #permet de conserver les bornes initiales

    while d-g>2*epsi:
        m=(g+d)/2.
        if f(g)*f(m) <= 0:
            d=m # on decale la borne de droite
        else:
            g=m # on decale la borne de gauche
    return m

def sol_newton(f,g,u0,epsi):
    u=u0

    while f(u)>=abs(eps):
        u=u-f(u)/g(u)
    return u

import numpy as np

tau1=int(input('tau1=?'))
tau2=int(input('tau2=?'))
epsi=float(input('epsi=?'))
t=3*(tau1+tau2)
a=t-0.1*t
b=t+0.1*t
f=lambda x:0.05-(tau1*np.exp(-x/tau1)-tau2*np.exp(-x/tau2))/(tau1-tau2)
g=lambda x:-(-np.exp(-x/tau1)+np.exp(-x/tau2))/(tau1-tau2)
td=sol_dicho(f,a,b,epsi)
tn=sol_newton(f,g,t,epsi)
print('Tr5%(dicho)=',td)
print('Tr5%(newton)=',tn)
```

2. DERIVATION NUMERIQUE

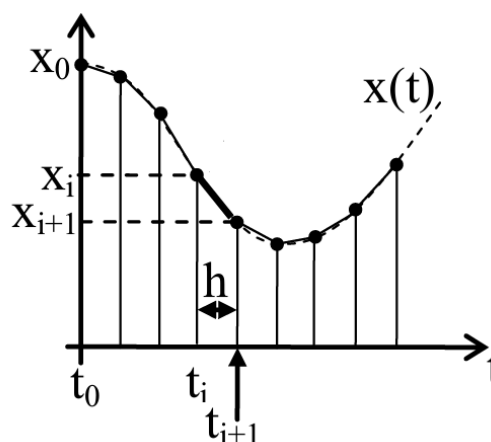
2.1 Dérivation numérique :

La méthode consiste à approximer une fonction par la méthode des accroissements finis. Pour ce faire, plusieurs approches sont possibles.

Soit pour un taux d'accroissement noté h :

- On utilise la valeur de la dérivée première arrière en t_k : $\frac{dx(t)}{dt} \simeq \frac{x_t - x_{t-h}}{h}$;
- On utilise la valeur de la dérivée première avant en t_k : $\frac{dx(t)}{dt} \simeq \frac{x_{t+h} - x_t}{h}$;
- On utilise la valeur de la dérivée seconde centrée en t_k : $\frac{d^2x(t)}{dt^2} \simeq \frac{x_{t+h} - 2x_t + x_{t-h}}{h^2}$;

On illustre ci-contre les valeurs prises par la dérivée première avant...



2.2 Proposition d'algorithme de dérivation sous python :

```
def derivee_1_avant(f,a,h):
    deriv1=(f(a+h)-f(a))/h
    return deriv1

def derivee_2(f,a,h):
    deriv2=(f(a+h)-2*f(a)+f(a-h))/h**2
    return deriv2
```

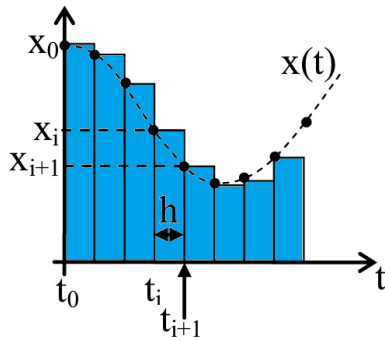
3. INTEGRATION NUMERIQUE

3.1 Principe :

On cherche à approximer l'intégrale $I = \int_a^b x(t)dt$.

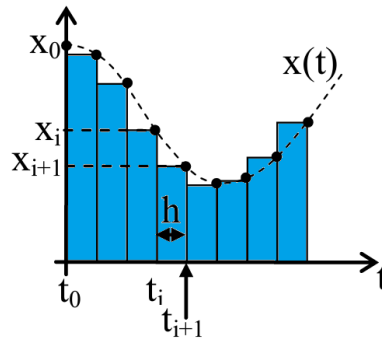
La méthode générale consiste à calculer l'aire sous la courbe $x(t)$. Pour cela, plusieurs solutions s'offrent à nous :

Rectangles à gauche



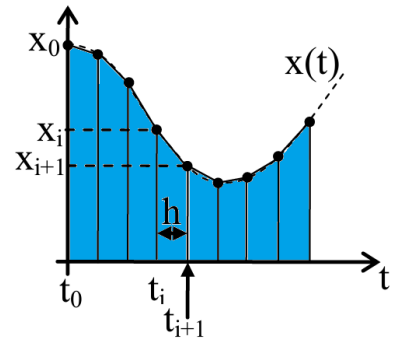
$$I \simeq h \sum_{i=0}^{n-1} x_i$$

Rectangles à droite



$$I \simeq h \sum_{i=1}^n x_i$$

Trapèzes



$$I \simeq h \left(\frac{x_0 + x_n}{2} + \sum_{i=1}^{n-1} x_i \right)$$

Les deux premières correspondent à des sommes de Riemann...

3.2 Proposition d'algorithme d'intégration sous python :

```
def rectangleg(f,a,b,n):#approximation a gauche
    t=a
    p=(b-a)/n
    s=f(t)
    for i in range(n-1):#attention au nb d iteration !!!
        t=t+p
        s=s+f(t)
    return(s*p)

def rectangled(f,a,b,n):#approximation a droite
    p=(b-a)/n
    t=a+p
    s=f(t)
    for i in range(n-1):
        t=t+p
        s=s+f(t)
    return(s*p)
```

La méthode des rectangles peut également se faire par le milieu...

```
def trapezes(f,a,b,n):
    t=a
    p=(b-a)/n
    s=(f(a)+f(b))/2

    for i in range(1,n):
        t=t+p
        s=s+f(t)
    return(s*p)
```

4. SCHEMA EXPLICITE D'EULER

4.1 Application à l'étude d'un 1^{er} ordre :

On rappelle les études en SII sont en général menées autour d'un point de fonctionnement, donc en considérant les **conditions initiales** comme **nulles**.

En conséquence, et si on prend une équation différentielle du 1^{er} d'une fonction $s(t)$ soumise à une entrée $e(t)$, on a :

$$s(t) + \tau \cdot \frac{ds(t)}{dt} = K \cdot e(t)$$

Qui peut s'écrire par l'approximation de la dérivée 1^{ère} avant :

$$S_{k+1} + \tau \cdot \frac{S_{k+1} - S_k}{h} = K \cdot E_k$$

Ou encore :

$$S_{k+1} = \frac{K \cdot h \cdot E_k + \tau \cdot S_k}{h + \tau}$$

4.3 Proposition d'algorithme d'approximation d'un 1^{er} ordre :

```
@author: Goncalves David
"""
def euler_1(f,k,tau,a,h,omega=1):
    n=int(a/h)
    s=0
    for i in range(n):
        s=(k*h*f(i*h*omega)+tau*s)/(h+tau)
    return s
```

Ici, l'argument optionnel ω sera utilisé pour une entrée $\sin(\omega \cdot t)$...

Algorithme qui renvoie pour une entrée $e(t) = f$ la solution approchée s par la méthode d'Euler.

```
import numpy as np
import matplotlib.pyplot as plt

k=float(input('k='))
tau=float(input('tau='))
n=int(input('n='))
t=float(input('t='))
f=eval(input('f='))
omega=float(input('omega='))
h=t/n

X=[h*i for i in range(n+1)]
Y=[euler_1(f,k,tau,i,h,omega) for i in X]
Y2=[f(omega*i) for i in X]

plt.clf()
plt.plot(X,Y,label='approchée',color='r')
plt.plot(X,Y2,label='entrée',color='b')
plt.legend()
plt.show()
```

Algorithme qui trace l'entrée $e(t) = f$ représentée par la liste $Y2$ et la solution approchée par la méthode d'Euler représentée par la liste Y jusque t pour $s(0) = 0$, à partir de la fonction `euler_1` !

4.3 Application à l'étude d'un 2nd ordre :

La méthode est identique :

$$s(t) + \frac{2\xi}{\omega_0} \cdot \frac{ds(t)}{dt} + \frac{1}{\omega_0^2} \cdot \frac{d^2s(t)}{dt^2} = K \cdot e(t)$$

Qui peut s'écrire par l'approximation de la dérivée 1^{ère} avant et dérivée seconde :

$$S_{k+1} + \frac{2\xi}{\omega_0} \cdot \frac{S_{k+1} - S_k}{h} + \frac{1}{\omega_0^2} \cdot \frac{S_{k+1} - 2S_k + S_{k-1}}{h^2} = K \cdot E_k$$

Ou encore :

$$S_{k+1} = \frac{K \cdot \omega_0^2 \cdot h^2 \cdot E_k + 2 \cdot S_k (\xi \cdot h \cdot \omega_0 + 1) - S_{k-1}}{\omega_0^2 \cdot h^2 + 2 \cdot \xi \cdot h \cdot \omega_0 + 1}$$

4.5 Proposition d'algorithme d'approximation d'un 2nd ordre :

```
@author: david.goncalves
"""
def euler_2(f,k,xsi,w0,h,a,omega=1):
    n=int(a/h)
    s2=0
    s1=0
    for i in range(n):
        s0=s1
        s1=s2
        s2=(k*w0**2*h**2*f(omega*i*h)+2*s1*(xsi*h*w0+1)-s0)/(w0**2*h**2+2*xsi*h*w0+1)
    return s1
```

Ici, l'argument optionnel omega sera utilisé pour une entrée sin(ω.t)...

Algorithme qui renvoie pour une entrée $e(t) = f$ la solution approchée s par la méthode d'Euler.

```
import numpy as np
import matplotlib.pyplot as plt

f=eval(input('e(t)='))
k=float(input('k='))
xsi=float(input('xsi='))
n=int(input('n='))
t=float(input('t='))
w0=float(input('w0='))
omega=float(input('omega='))

h=t/n
X=[i*h for i in range(n+1)]
Y=[euler_2(f,k,xsi,w0,h,i,omega) for i in X]
Y2=[f(omega*i) for i in X]

plt.clf()
plt.plot(X,Y,label='approchée',color='r')
plt.plot(X,Y2,label='entrée',color='b')
plt.legend()
plt.show()
```

Algorithme qui trace l'entrée $e(t) = f$ représentée par la liste $Y2$ et la solution approchée par la méthode d'Euler représentée par la liste Y jusque t pour $s(0) = 0$, à partir de la fonction `euler_2` !